BUỔI 2

Loss, backpropagation, tối ưu hóa và quy trình huấn luyện MLP

- Hiểu vai trò của loss, gradient, backpropagation và optimizer.
- Viết được training loop PyTorch cơ bản.
- Biết phân tích lỗi huấn luyện qua loss/accuracy.
- Thực hành MLP trên FashionMNIST.

Loss, back propagation và huấn luyện MLP



VÍ DỤ THỰC TẾ

Phân loại 10 loại trang phục: cho mô hình xem ảnh và nhãn đúng, rồi học cách giảm sai số dự đoán.

1. Hàm mất mát: mô hình sai bao nhiêu?

- Loss biến chất lượng dự đoán thành một con số có thể tối ưu.
- Bài toán phân loại đa lớp thường dùng cross-entropy loss.
- Bài toán hồi quy thường dùng MSE hoặc MAE tùy mục tiêu.
- Loss thấp trên train nhưng cao trên validation là dấu hiệu cần kiểm tra generalization.
- Không nên tối ưu metric bằng cảm tính, cần chọn loss tương thích với đầu ra.

Trong bài toán phân loại nhiều lớp, mô hình dự đoán xác suất cho từng lớp. Negative Log-Likelihood (NLL) và Cross-Entropy (CE) đo **mức sai lệch giữa xác suất dự đoán và nhãn đúng**.

- NLL: Lấy âm logarit của xác suất mà mô hình gán cho lớp đúng.
- Cross-Entropy: Đo sự khác biệt giữa phân phối xác suất dự đoán và phân phối nhãn thật. Với nhãn one-hot (chỉ có 1 giá trị bằng 1, còn lại là 0), CE chính là NLL.

Công thức:

$$L = -\log(p_y)$$

Trong đó p_y là xác suất mô hình dự đoán cho lớp đúng y.

Ví dụ: Phân loại ảnh thành 3 lớp: mèo, chó, chim. Nhãn đúng là *mèo*.

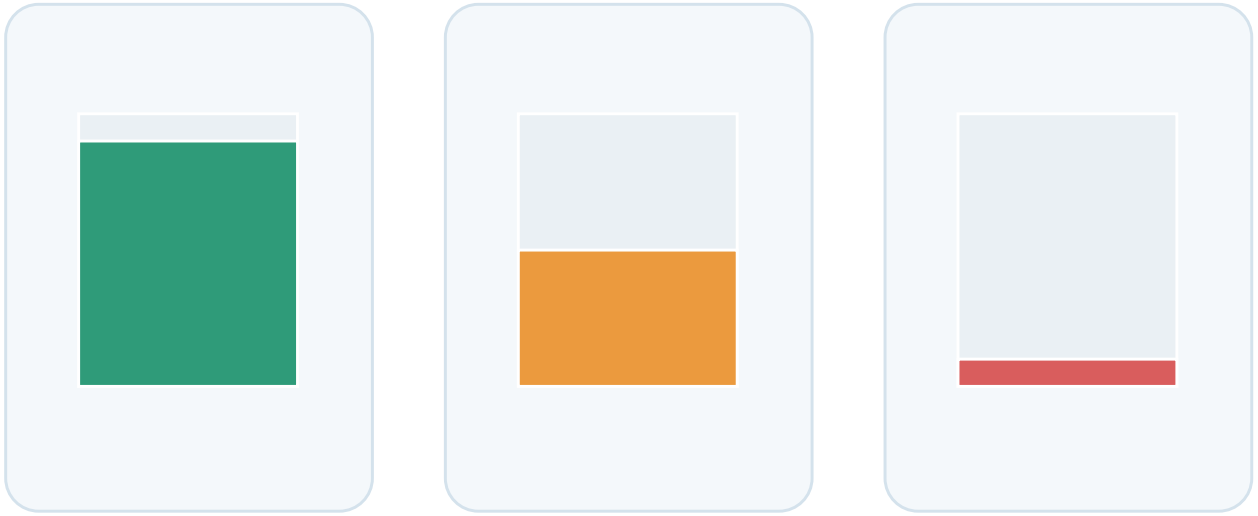
- Dự đoán `[0.9, 0.08, 0.02]` → Loss = $-\ln(0.9) \approx 0.105$ (tốt).
- Dự đoán `[0.1, 0.8, 0.1]` → Loss = $-\ln(0.1) \approx 2.303$ (kém).

Kết luận: Mô hình càng gán xác suất cao cho lớp đúng thì loss càng nhỏ. Trong PyTorch, nn.CrossEntropyLoss() nhận trực tiếp logits, không cần áp dụng Softmax trước.

MINH HỌA • Hàm mất mát: mô hình sai bao nhiêu?

SƠ ĐỒ / HÌNH MINH HỌA

Dự đoán đúng càng ít tự tin → loss càng lớn



p đúng=0.9	p đúng=0.5	p đúng=0.1
Loss=0.11	Loss=0.69	Loss=2.30

VÍ DỤ THỰC TẾ

Ảnh một chiếc áo bị đoán nhầm thành túi: loss càng lớn khi mô hình càng tự tin vào dự đoán sai.

ĐỘ TỰ TIN (CONFIDENCE) CỦA MÔ HÌNH

Ví dụ: Mạng nơ-ron phân loại một ảnh thành Mèo – Chó – Chim

Tính độ tự tin qua 3 bước

1. Mạng tạo ra điểm số thô (logits)

Mèo: 0,0 Chó: 2,1 Chim: −1,0

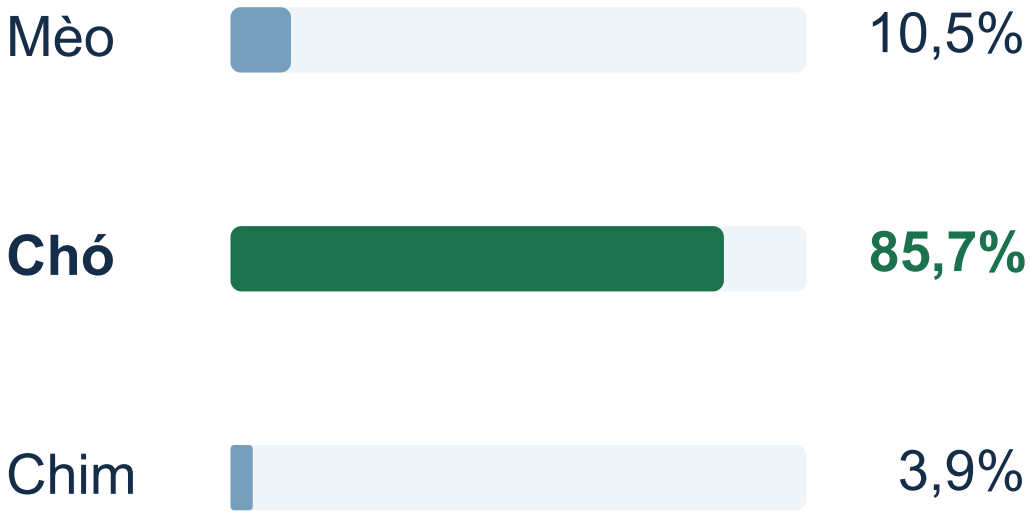
2. Softmax đổi điểm số thành xác suất

$$P(k) = e^{zk} / \Sigma_j e^{zj}$$

3. Lấy xác suất lớn nhất

Confidence = 85,7% → CHÓ

Xác suất sau Softmax



Lưu ý: 85,7% là mức tự tin của mô hình, không phải bảo đảm ảnh đúng là Chó.
Cần kiểm tra trên tập dữ liệu độc lập để biết mức tự tin có đáng tin cậy không.

$$\boxed{P(k) = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}}$$

Trong đó:

- $P(k)$: Xác suất dự đoán của lớp k.
- z_k: Điểm số thô (logit) của lớp k.
- K: Tổng số lớp cần phân loại.
- e: Hằng số Euler ($\approx 2{,}718$).
- $\sum_{j=1}^{K} e^{z_j}$: Tổng các giá trị mũ của logits ở tất cả các lớp.

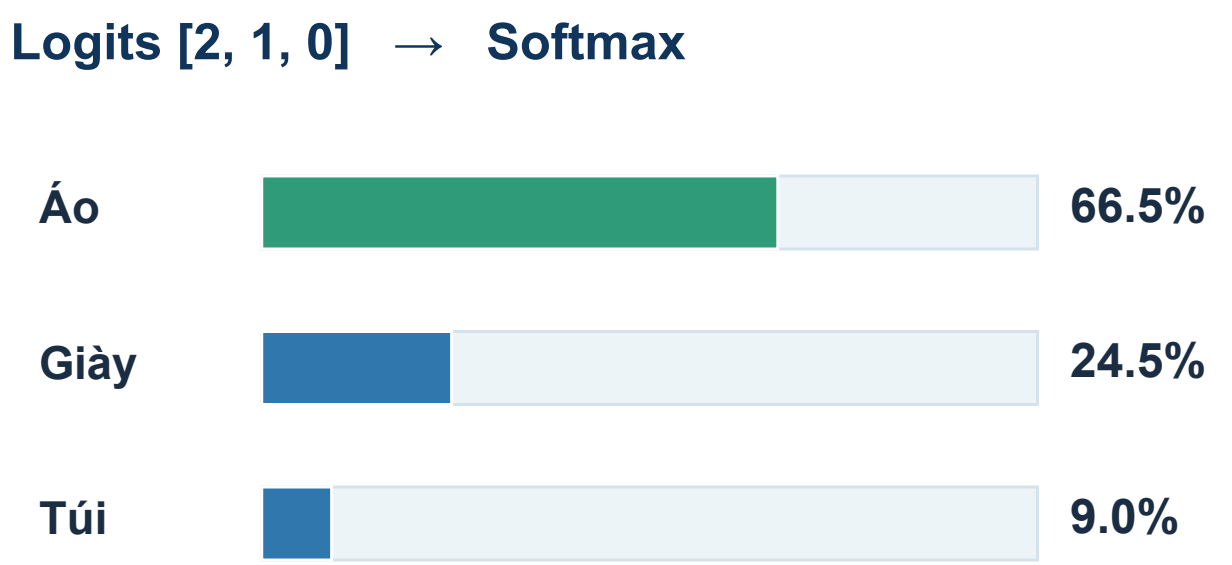
Đây là **công thức Softmax**, dùng để chuyển logits thành xác suất dự đoán cho từng lớp.

Cross-entropy

- Mô hình xuất logits cho từng lớp; softmax chuyển logits thành phân phối xác suất.
- Cross-entropy phạt mạnh khi mô hình tự tin vào lớp sai.
- Trong PyTorch, CrossEntropyLoss nhận logits thô, không cần softmax trước.
- Nhãn y thường là chỉ số lớp, không phải one-hot vector.
- Sai kiểu dữ liệu nhãn là lỗi thường gặp: y cần dạng LongTensor.

MINH HỌA • Cross-entropy

SƠ ĐỒ / HÌNH MINH HỌA



VÍ DỤ THỰC TẾ

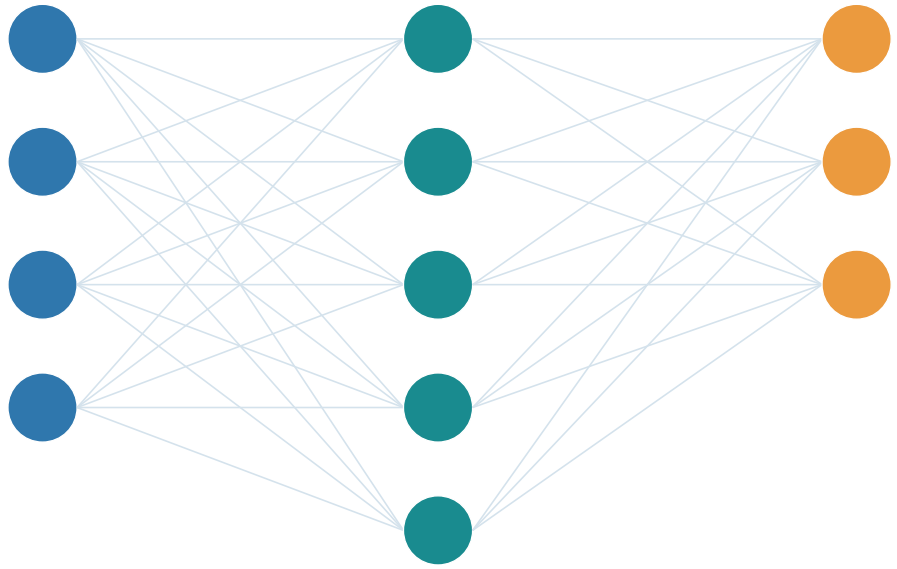
Nhãn đúng là “Áo”: logits [2,1,0] cho xác suất Áo ≈ 66,5%, dẫn tới CrossEntropyLoss ≈ 0,41.

Computational graph và autograd

- Mỗi phép tính tensor có thể tạo một nút trong computational graph.
- Autograd tự động tính gradient của loss theo tham số có requires_grad=True.
- Quy trình chuẩn: zero_grad → forward → loss → backward → optimizer.step.

MINH HỌA • Computational graph và autograd

SƠ ĐỒ / HÌNH MINH HỌA



Lan truyền tiến → tính loss → lan truyền ngược

VÍ DỤ THỰC TẾ

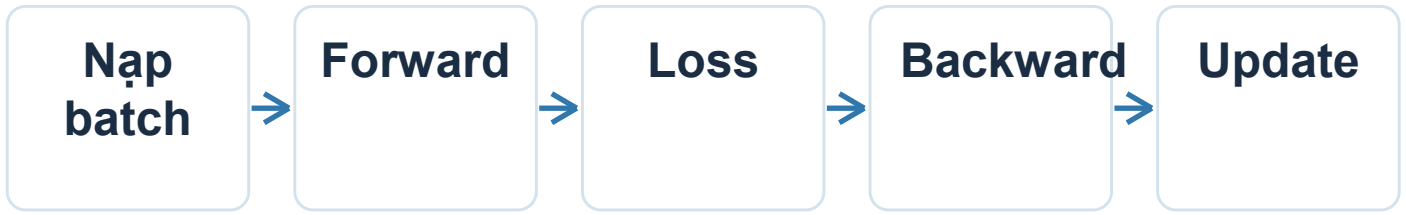
Trong một MLP nhận ảnh 28×28, autograd tính độ thay đổi loss đối với từng trọng số.

Training loop

- Mỗi epoch lặp qua nhiều mini-batch.
- Với mỗi mini-batch: đưa x,y vào device; forward; tính loss; backward; cập nhật tham số.
- Sau mỗi epoch, đánh giá validation ở chế độ model.eval().
- Lưu train loss, validation loss/accuracy để phân tích learning curve.
- Không đánh giá mô hình trên test set trong quá trình chọn hyperparameter.

MINH HỌA • Training loop

SƠ ĐỒ / HÌNH MINH HỌA



Đầu vào

Đầu ra / kết quả

VÍ DỤ THỰC TẾ

Sau mỗi mini-batch ảnh quần áo, cập nhật trọng số; hết một epoch thì kiểm tra trên tập validation.

Gradient descent và mini-batch

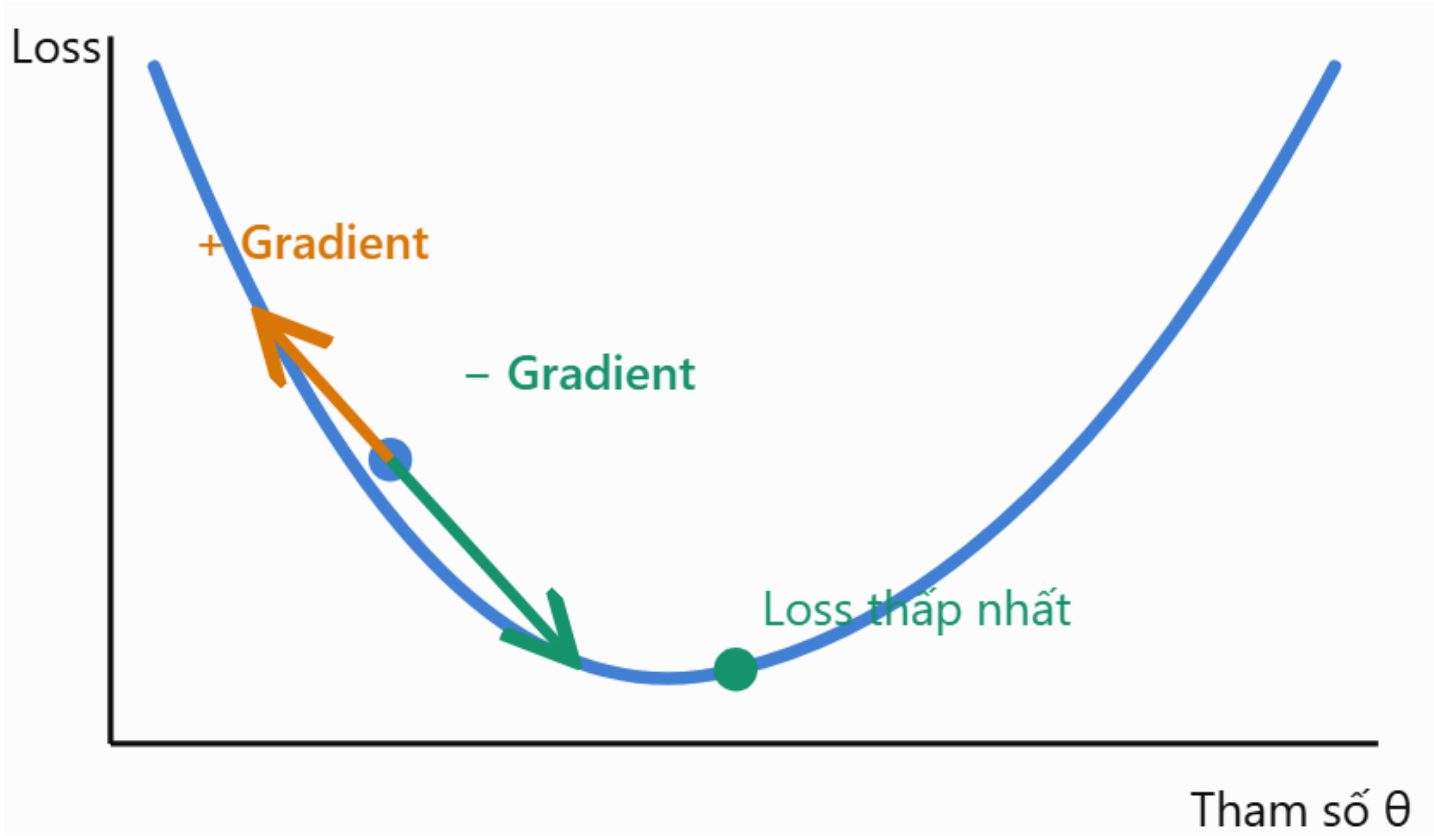
- Gradient chỉ hướng thay đổi loss nhanh nhất theo tham số.
- Gradient descent cập nhật $\theta \leftarrow \theta - \eta \nabla_\theta L$.
- η là learning rate; quá lớn có thể dao động, quá nhỏ học rất chậm.
- Mini-batch cân bằng giữa tốc độ, bộ nhớ và độ ổn định gradient.
- Adam thường dễ dùng ban đầu; SGD giúp hiểu rõ vai trò learning rate.

1.1. Gradient là gì?

Gradient (đạo hàm riêng theo các tham số) cho biết hai thông tin quan trọng:

- Hướng: **Loss tăng nhanh nhất** khi **thay đổi các tham số theo hướng gradient**.
- Độ lớn: **Gradient lớn** thường cho thấy **loss thay đổi mạnh theo tham số**; **gradient nhỏ cho thấy loss thay đổi ít**.

Do mục tiêu huấn luyện là giảm loss, chúng ta **cập nhật tham số theo hướng ngược gradient**.



Minh họa trường hợp một tham số: đi ngược gradient để giảm loss.

Ví dụ: Khi điều chỉnh trọng số của mạng neural, gradient giúp xác định nên tăng hay giảm trọng số đó để giảm sai số dự đoán.

1.2. Gradient Descent hoạt động như thế nào?

Công thức cập nhật:

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta L(\theta_t)$$

Ký hiệu	Ý nghĩa
θ_t	Tham số hiện tại (weights, biases)
L	Hàm mất mát (Loss function)
$\nabla_\theta L$	Gradient của loss theo tham số
η	Learning rate – tốc độ học
θ_{t+1}	Tham số sau cập nhật

1.3. Learning rate ảnh hưởng như thế nào?



Learning rate giống độ dài mỗi bước chân khi đi xuống thung lũng. Bước quá nhỏ thì đi chậm; bước quá dài có thể vượt qua đáy thung lũng

1.4. Mini-batch là gì?

Giả sử tập huấn luyện có **1.000 mẫu**. Có ba cách tính gradient và cập nhật tham số phổ biến:

Phương pháp	Số mẫu mỗi lần tính gradient	Số lần cập nhật/epoch
Batch Gradient Descent	1.000	1
Stochastic GD (SGD một mẫu)	1	1.000
Mini-batch GD (batch size = 100)	100	10

Mini-batch chia dữ liệu thành các nhóm nhỏ và dùng mỗi nhóm để ước lượng gradient, thay vì sử dụng toàn bộ dữ liệu ở mỗi lần cập nhật.

Ví dụ với 1.000 mẫu, batch size = 100:

- Một epoch xử lý đủ 1.000 mẫu.
- Có 10 mini-batch.
- Mỗi mini-batch thực hiện một lần cập nhật tham số.
- Sau 20 epoch sẽ có 200 lần cập nhật (nếu không bỏ batch nào).

Ưu điểm: tiết kiệm bộ nhớ so với full-batch, tận dụng xử lý song song GPU và thường có tốc độ huấn luyện tốt.
Cần phân biệt: Batch size không phải số epoch. Batch size quyết định số mẫu trong một bước; epoch là một lượt duyệt toàn bộ tập huấn luyện.

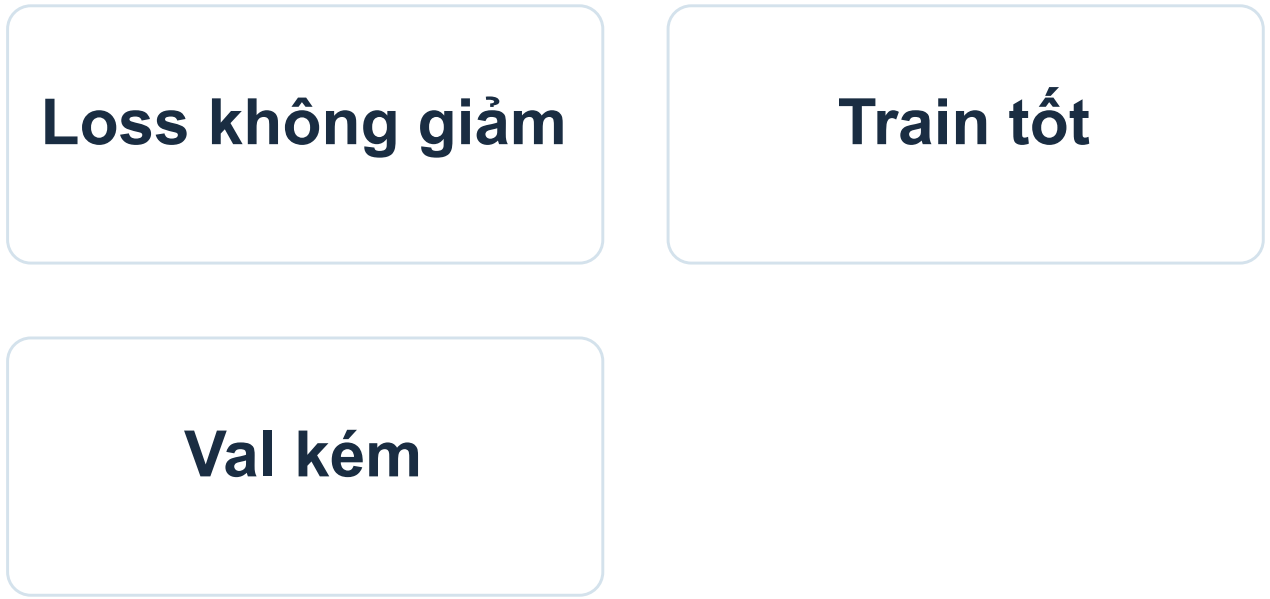
- Gradient xác định hướng tăng nhanh nhất của loss; Gradient Descent đi theo hướng ngược lại.
- Learning rate quyết định kích thước bước cập nhật; mini-batch quyết định số mẫu dùng để ước lượng gradient trong mỗi bước.
- SGD và Adam là các thuật toán tối ưu; mini-batch là cách tổ chức dữ liệu để tính gradient. Hai khái niệm này không đối lập nhau.

Tối ưu hóa không chỉ là “chạy nhiều epoch”

- Nếu loss không giảm: kiểm tra dữ liệu, label, learning rate và architecture.
- Nếu train tốt nhưng validation kém: cân nhắc regularization, augmentation hoặc giảm capacity.
- Nếu accuracy dao động mạnh: batch size nhỏ, learning rate cao hoặc dữ liệu nhiễu.
- Nếu GPU không hoạt động: kiểm tra device của cả model và tensor.
- Mỗi thí nghiệm phải có baseline để so sánh.

MINH HỌA • Tối ưu hóa không chỉ là “chạy nhiều epoch”

SƠ ĐỒ / HÌNH MINH HỌA



VÍ DỤ THỰC TẾ

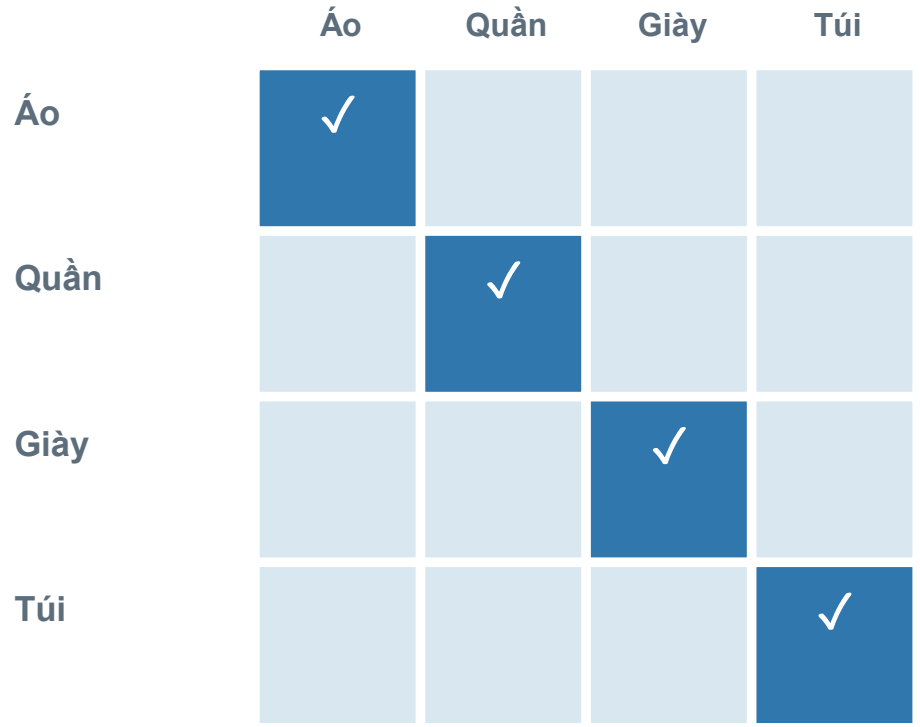
Mô hình dự đoán toàn một loại áo: kiểm tra nhãn, learning rate và bước optimizer trước khi sửa kiến trúc.

Đánh giá MLP

- Accuracy dễ hiểu nhưng chưa đủ khi dữ liệu lệch lớp.
- Confusion matrix cho biết lớp nào hay bị nhầm với lớp nào.
- Validation set dùng để chọn mô hình; test set chỉ dùng cho đánh giá cuối.
- Báo cáo cần ghi rõ split, số epoch, batch size, optimizer và learning rate.
- Khi so sánh mô hình, chỉ thay một yếu tố chính tại một thời điểm.

MINH HỌA • Đánh giá MLP

SƠ ĐỒ / HÌNH MINH HỌA



Hàng: nhãn đúng | Cột: nhãn dự đoán

VÍ DỤ THỰC TẾ

Trong 100 ảnh áo khoác, mô hình nhầm 25 ảnh thành áo sơ mi; confusion matrix chỉ ra lỗi này.

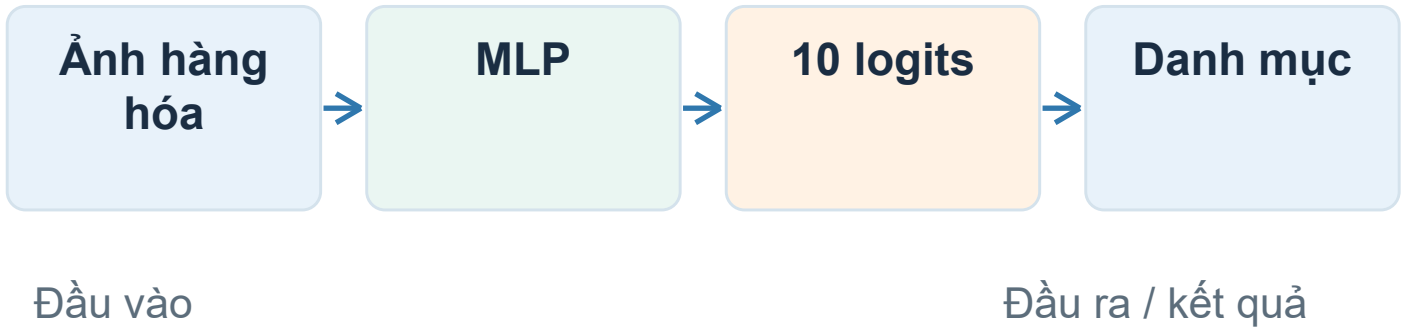
Chỉ số/số liệu minh họa để giải thích trực giác.

Ví dụ thực tế: phân loại sản phẩm thời trang

- FashionMNIST gồm ảnh 28×28 thuộc 10 loại sản phẩm như áo, giày, túi.
- MLP dùng flatten ảnh nên bỏ qua cấu trúc không gian của pixel.
- Nếu MLP nhầm áo sơ mi với áo thun, cần xem ảnh mẫu và confusion matrix.
- Đây là cầu nối sang CNN: ảnh có cấu trúc lân cận mà MLP chưa khai thác tốt.
- Câu hỏi: vì sao hoán vị pixel làm MLP và CNN bị ảnh hưởng khác nhau?

MINH HỌA • Ví dụ thực tế: phân loại sản phẩm thời trang

SƠ ĐỒ / HÌNH MINH HỌA



VÍ DỤ THỰC TẾ

Website bán quần áo phân loại ảnh đăng bán thành áo, váy, túi hoặc giày để hỗ trợ gắn danh mục.

Thực hành 1: Xây dựng Training Loop

Mục tiêu: Hoàn thiện quy trình huấn luyện và đánh giá mô hình MLP trên tập con FashionMNIST.

Yêu cầu:

- Sử dụng file gợi ý bai02_class_training_loop.py.
- Xây dựng vòng lặp huấn luyện đầy đủ: zero_grad → forward → loss → backward → optimizer.step.
- In thiết bị sử dụng (CPU/GPU), tổng số tham số, train loss và validation accuracy sau mỗi epoch.
- Thử ba learning rate: 0.01; 0.001; 0.0001, giữ nguyên các cấu hình khác; so sánh tốc độ hội tụ và kết quả.
- Sử dụng đúng model.train() khi huấn luyện và model.eval() khi đánh giá.

Sản phẩm nộp: Mã nguồn Python chạy được; kết quả huấn luyện (device, số tham số, train loss và validation accuracy mỗi epoch); bảng so sánh 3 learning rate (0.01; 0.001; 0.0001) và nhận xét ngắn về tốc độ hội tụ.

Thực hành 2: So sánh các phương pháp tối ưu

Mục tiêu: Đánh giá ảnh hưởng của optimizer đến quá trình huấn luyện MLP trên FashionMNIST.

Yêu cầu theo mức độ:

- Cơ bản: Huấn luyện cùng một kiến trúc MLP bằng SGD và Adam; sử dụng cùng dữ liệu, số epoch và báo cáo learning rate.
- Khá: Vẽ biểu đồ train loss và validation loss theo epoch cho từng optimizer.
- Nâng cao: Cài đặt Early Stopping khi validation loss không cải thiện sau một số epoch liên tiếp.

Sản phẩm nộp: Mã nguồn chạy được, bảng so sánh cấu hình và kết quả, biểu đồ learning curve, nhận xét ưu/nhược điểm của SGD và Adam, nguyên nhân lỗi và hướng cải thiện.